

LUCRAREA 7

Lucrul în intreruperi cu microcontrolerul FREESCALE ARM Cortex M0+

1. OBIECTUL LUCRĂRII

Lucrarea isi propune studierea si insusirea notiunilor teoretice, precum si rezolvarea unor probleme concrete legate de lucrul in intreruperi folosind circuitele I/O ale microcontrolerului Freescale Cortex M0+.

2. BREVIAR TEORETIC

În scrierea programelor apar situații în care un program conține mai multe părți ce necesită o execuție simultană. Programarea acestor tipuri de aplicații este o programare paralelă sau concurentă. Orice aplicație în timp real conține de obicei astfel de operații ce se derulează paralel. Un program ce se execută în timp real trebuie să răspundă practic instantaneu la semnalele ce se aplică din exterior, iar “timpul real” în care sistemul de calcul este ocupat cu o anumită operație trebuie să fie strâns corelat cu timpul în care se produc alte evenimente, în vederea tratării tuturor acestora. Un program ce realizează coordonarea mai multor activități în același timp se cheamă cu facilități de tip multitasking. Lucrul în întreruperi prezintă avantajul faptului că permite procesorului gestionarea mai multor activități, practic simultan. Sunt mai multe tehnici prin care acest lucru este posibil.

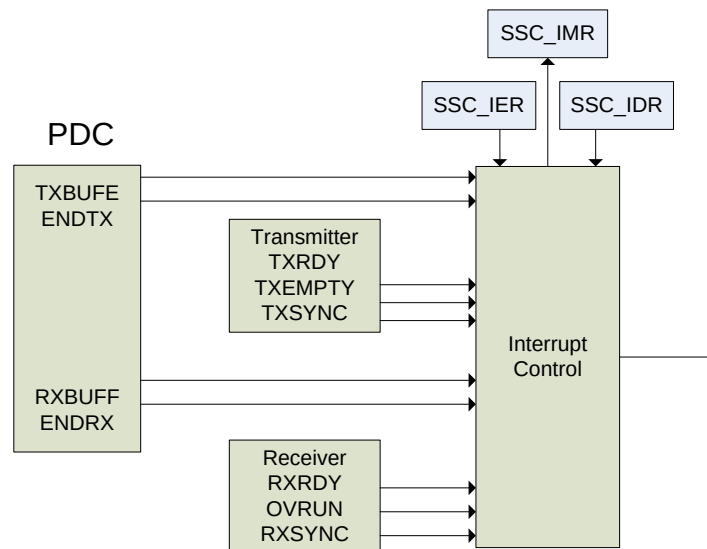
O tehnică implică gestionarea unor activități în programul principal, iar apariția altor evenimente (activități sau task-uri) ce necesită tratare, este semnalizată procesorului prin cereri de întrerupere. În aceste situații, procesorul suspendă activitățile în curs din programul principal și trece la tratarea evenimentelor ce au survenit. După tratarea acestora, revine în programul principal în locul în care a apărut suspendarea. Este posibilă chiar suspendarea tratării unei cereri de întrerupere în situația apariției unei a doua apărute cu grad crescut de prioritate (întreruperi imbricate).

O a doua tehnică presupune alocarea unui timp de tratare bine precizat (egal sau diferit) fiecărei activități ce trebuie gestionată (time sharing sau time slicing).

În ambele situații, viteza mare de execuție a procesorului lasă impresia că task-urile se

execută simultan, ca și când fiecare task deține controlul. În acest caz se spune că are loc o execuție pseudo-paralelă sau execuție paralelă logică.

Diagrama bloc a intreruperilor



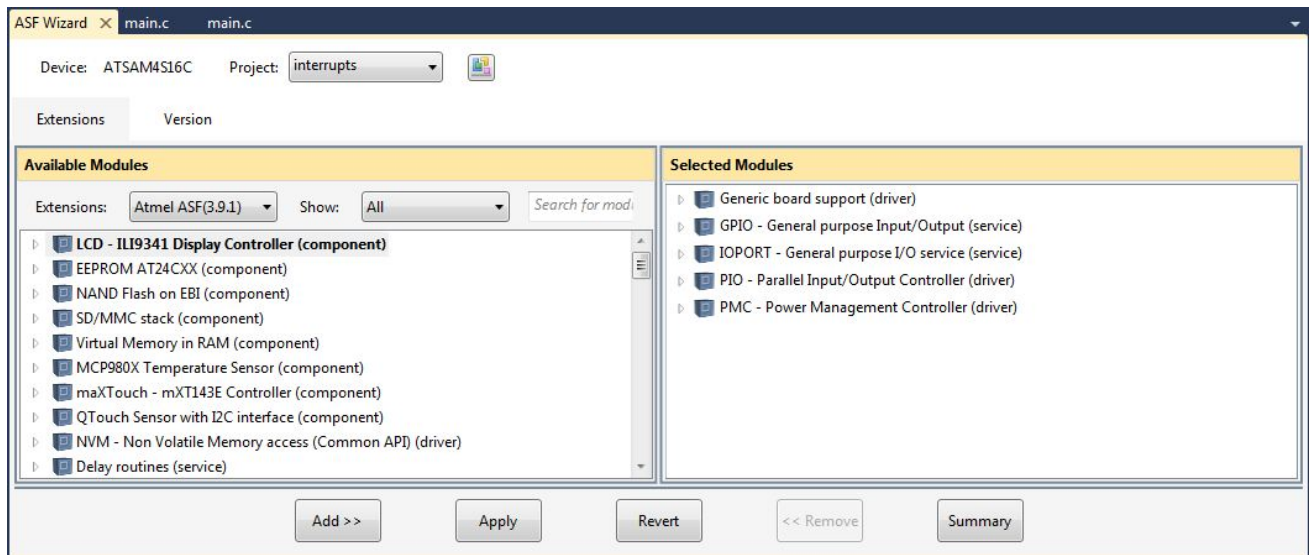
Initializari

Înainte de initializarea pinilor va trebui să adăugăm din ASF (Atmel Software Library) modulele de realizare a intreruperilor și modulul de clock al sistemului. Pentru a adăuga modulele se procedează astfel:

Dăți click pe ASF din bara de instrumente și selectați ASF Wizard, sau puteți da click pe

icon-ul  din bara de instrumente.

Următoarea fereastră se va deschide:



Modulele din partea dreapta reprezinta modulele care au fost adaugate initial la crearea proiectului. In partea stanga avem lista din care putem alege module noi. Vom avea nevoie de modulul numit Interrupt management (Common API (Driver)) si de modulul System Clock Control (Service). Dupa ce a-ti dat click pe Add >> pentru selectarea modulelor se apasa butonul Apply pentru a aplica modulele la proiectul activ pe care se lucreaza. Se observa ca modulul pentru intreruperi este de fapt un driver iar modulul pentru clock este un serviciu.

Pentru vizualizarea modulelor in timpul programarii puteti deschide tab-ul ASF Explorer folosind combinatia de taste Alt+A.

Dupa adaugarea modulelor necesare se poate trece la scrierea codului pentru initializare.

Analiza codului de initializare

```
board_init();
```

```
PMC->PMC_PCER0 = ID_PIOA;
```

```
PIOA->PIO_PUER = PIO_PA5;
```

```
PIOC->PIO_PER=PIO_PC10;
```

```
PIOC->PIO_OER=PIO_PC10;
```

```
PIOA->PIO_PER=PIO_PA5;
```

```
PIOA->PIO_IFER=PIO_PA5;
```

```
pio_handler_set(PIOA, ID_PIOA, PIO_PA5, PIO_IT_EDGE, pin_handler);
```

PIOA->PIO_IER=PIO_PA5;

NVIC->ISER[0] =(1 << PIOA_IRQn);

In primul rand pentru functionarea butonului va trebui sa activam un semnal de ceas pentru respectivul buton deoarece procesorul verifica starea butonului la fiecare schimbare de front a ceasului. Daca nu adaugam nici un semnal de ceas procesorul nu va raspunde la comanda data de la buton pentru ca nu ar putea sa ii citeasca schimbarea de stare.

Initializari clock

PMC->PMC_PCER0 = ID_PIOA;

PMC - Peripheral Main Clock

PMC_PCER0 = ID_PIOA - Activeaza clock-ul pentru portul A (Peripheral Clock Enable Register).

In continuare va trebui sa alegem modul de lucru al butonul si anume frontul pe care va activa secventa de cod pe care o vom scrie.

PIOA->PIO_PUER = PIO_PA5;

Aceasta comanda va activa registrul de pull-up al pinului 5 de pe portul A.

PIO_PUER - Pull-Up Enable Register.

Initializari pini led

In continuare se activeaza pinul la care este conectat ledul:

PIOC->PIO_PER=PIO_PC10;

PIOC->PIO_OER=PIO_PC10;

Prima comanda va activa registrul ce contine adresa pinului 10 al portului C.

PIO_PER - Pin Enable Register.

A doua comanda seteaza pinul 10 din portul C ca fiind pin de iesire.

PIO_OER - Output Enable Register.

Initializari pini buton

Asemanator activarii ledului se procedeaza si cu butonul:

PIOA->PIO_PER=PIO_PA5;

PIOA->PIO_IFER=PIO_PA5;

Prima comanda activeaza registrul ce contine adresa pinului 5 al portului A.

A doua comanda activeaza filtrul pentru intreruperi pentru pinul 5 din portul A.

PIO_IFER - Interrupt Filter Enable Register.

Initializari handler intreruperi

Pentru functionarea intreruperilor este nevoie de scrierea unei functii pe care sa o apelam in momentul activarii intreruperii.

pio_handler_set(PIOA, ID_PIOA, PIO_PA5, PIO_IT_EDGE, pin_handler);

Folosind functia *pio_handler_set()* furnizata de librariile Atmel, putem apela functia *pin_handler* creata de noi ce va fi apelata in momentul in care procesorul detecteaza o schimbare de front (*PIO_IT_EDGE*) pe pinul 5 din portul A.

Urmatoarea comanda este aceea de a activa intreruperile pentru pinul 5 (butonul):

PIOA->PIO_IER=PIO_PA5;

PIO_IER - Interrupt Enable Register.

Manevrarea intreruperilor necesita programarea NVIC (Nested Vectored Interrupt Controller).

Nested Vectored Interrupt Controller (NVIC)

NVIC suporta:

- 1 pana la 35 intreruperi.
- Un nivel programabil (0-15) de prioritati pentru fiecare intrerupere. Un nivel mai inalt reprezinta o prioritate mai scazuta, asadar nivelul 0 este cel mai inalt nivel de prioritate.
- Nivel de detectie pentru semnalele de intrerupere.
- Reprioritizarea dinamica a intreruperilor.

Pentru a programa NVIC folosim comanda:

```
NVIC->ISER[0] =(1 << PIOA_IRQn);
```

ISER[0] - Interrupt Set Enable Register 0.

Aceasta instructiune programeaza NVIC astfel incat sa astepte intreruperea ce va veni de pe portul A.

Functia *pin_handler()*

In aceasta functie vom putea scrie programul pe care dorim sa il rulam in momentul in care se activeaza intreruperea.

Aceasta functie nu va trebui apelata niciodata in programul principal pentru ca generarea intreruperii va face acest lucru pentru noi.

3. Probleme rezolvate

1. Sa se realizeze un program ce va genera intrerupere la fiecare apasare de buton. Cu fiecare intrerupere se va aprinde unul din cele 2 leduri de pe placa de dezvoltare.

```
#include <asf.h>
```

```
void pin_handler(void);
```

```
int main (void)
```

```
{  
    board_init();  
    PMC->PMC_PCER0 = ID_PIOA;  
    PIOA->PIO_PUER = PIO_PA5;  
    PIOC->PIO_PER=PIO_PC10;  
    PIOC->PIO_OER=PIO_PC10;
```

```

    PIOA->PIO_PER=PIO_PA5;
    PIOA->PIO_IFER=PIO_PA5;
    pio_handler_set(PIOA, ID_PIOA, PIO_PA5, PIO_IT_EDGE, pin_handler);
    PIOA->PIO_IER=PIO_PA5;
    NVIC->ISER[0] =(1 << PIOA_IRQn);
    while (true)
    {
    }
}
void pin_handler(void)
{
    if (PIOA->PIO_PDSR & (PIO_PA5))
        PIOC->PIO_SODR=PIO_PC10;
    else
        PIOC->PIO_CODR=PIO_PC10;
}

```

2. Realizati un program care va tine aprins ledul conectat la pinul 10 al portului C pana cand se va genera intrerupere moment in care ledul de pe PC10 se va stinge si ledul de pe PC17 se va aprinde.

```
#include <asf.h>
```

```
void pin_handler(void);
```

```
int main (void)
```

```

{
    board_init();
    PMC->PMC_PCER0 = ID_PIOA;
    PIOA->PIO_PUER = PIO_PA5;
    PIOC->PIO_PER=PIO_PC10;
    PIOC->PIO_OER=PIO_PC10;
    PIOA->PIO_PER=PIO_PA5;
}

```

```

    PIOA->PIO_IFER=PIO_PA5;
    pio_handler_set(PIOA, ID_PIOA, PIO_PA5, PIO_IT_EDGE, pin_handler);
    PIOA->PIO_IER=PIO_PA5;
    NVIC->ISER[0] =(1 << PIOA_IRQn);
    while (true)
    {
    }
}

void pin_handler(void)
{
    if (PIOA->PIO_PDSR & (PIO_PA5))
    {
        PIOC->PIO_SODR=PIO_PC17;
        PIOC->PIO_CODR=PIO_PC10;
    }
    else
    {
        PIOC->PIO_CODR=PIO_PC17;
        PIOC->PIO_SODR=PIO_PC10;
    }
}

```

3. Sa se realizeze o aplicatie ce va permite aprinderea unui led sa se aprinda la apasarea butonului si sa se stinga la a doua apasare.

```
#include <asf.h>
```

```

void pin_handler(void);
void check();
int but;
int ok = 0;

```



```
int main (void)
{
    board_init();
    PMC->PMC_PCER0 = ID_PIOA;
    PIOA->PIO_PUER = PIO_PA5;
    PIOC->PIO_PER=PIO_PC10;
    PIOC->PIO_OER=PIO_PC10;
    PIOA->PIO_PER=PIO_PA5;
    PIOA->PIO_IFER=PIO_PA5;
    pio_handler_set(PIOA, ID_PIOA, PIO_PA5, PIO_IT_EDGE, pin_handler);
    PIOA->PIO_IER=PIO_PA5;
    NVIC->ISER[0] =(1 << PIOA_IRQn);
    while (true)
    {
    }
}

void pin_handler(void)
{
    check();
    if (but == 0)
        if (ok == 0)
        {
            PIOC->PIO_SODR=PIO_PC10;
            ok = 0;
        }
        else
        {
            PIOC->PIO_CODR=PIO_PC10;
            ok = 1 ;
        }
    check();
    if (but == 1)
        if (ok == 1)
```

```

        {
            PIOC->PIO_CODR=PIO_PC10;
            ok = 0;
        }
        else
        {
            PIOC->PIO_SODR=PIO_PC10;
            ok = 1;
        }
    }
    void check()
    {
        if (PIOA->PIO_PDSR & (PIO_PA5))
            but = 1;
        else but = 0;
    }

```

Observati ca nu se mai folosesc intarzieri sau verificari duble ale butonului, deoarece in momentul apasarii butonului se genereaza intrerupere, urmatoarea intrerupere se va genera numai la aparitia urmatorului front cazator al butonului. Putem spune ca aceasta metoda este una eficienta de inlocuire a debouncing-ului, pentru ca nu se mai folosesc bucle de intarziere ce ingreuneaza procesorul.

Am creat functia check() pentru o verificare rapida a starii butonului, si pentru simplificarea si minimizarea codului.

4. Realizati o aplicatie asemanatoare celei anterioare, care va tine un led aprins si unul stins, iar la o apasare de buton se vor inversa starile butoanelor.

```

#include <asf.h>
void pin_handler(void);
void check();
int but;
int ok = 0;

```

```
int main (void)
{
    board_init();

    PMC->PMC_PCER0 = ID_PIOA;
    PIOA->PIO_PUER = PIO_PA5;
    PIOC->PIO_PER=PIO_PC10;
    PIOC->PIO_OER=PIO_PC10;
    PIOA->PIO_PER=PIO_PA5;
    PIOA->PIO_IFER=PIO_PA5;
    pio_handler_set(PIOA, ID_PIOA, PIO_PA5, PIO_IT_EDGE, pin_handler);
    PIOA->PIO_IER=PIO_PA5;
    NVIC->ISER[0] =(1 << PIOA_IRQn);
    while (true)
    {
    }
}

void pin_handler(void)
{
    check();
    if (but == 0)
        if (ok == 0)
        {
            PIOC->PIO_SODR=PIO_PC10;
            PIOC->PIO_CODR=PIO_PC17;
            ok = 0;
        }
        else
        {
            PIOC->PIO_CODR=PIO_PC10;
            PIOC->PIO_SODR=PIO_PC17;
            ok = 1 ;
        }
}
```

```

    check();
    if (but == 1)
        if (ok == 1)
        {
            PIOC->PIO_CODR=PIO_PC10;
            PIOC->PIO_SODR=PIO_PC17;
            ok = 0;
        }
        else
        {
            PIOC->PIO_SODR=PIO_PC10;
            PIOC->PIO_CODR=PIO_PC17;
            ok = 1;
        }
    }
void check()
{
    if (PIOA->PIO_PDSR & (PIO_PA5))
        but = 1;
    else but = 0;
}

```

4. Desfasurarea lucrarii

1. Realizati o aplicatie care va aprinde ambele leduri la apasarea butonului si stingerea acestora la eliberarea butonului.
2. Ce rol are comanda `PIOA->PIO_PDSR & (PIO_PA5)` din functia `check()`?
3. Construiti o functie separata ce va numara apasarile de buton, cu fiecare apasare frecventa de licarire a ledului se va dubla.
4. Realizati o aplicatie ce va porni licarirea unui led prin apasare de buton, la a doua apasare oprind licarirea.
5. Explicati modul general de functionare a intreruperilor.
6. Realizati organigramele de functionare pentru problemele rezolvate